

CareerForge Pro

ATS-Proof Resume Generator & Job Matcher

ENTERPRISE ARCHITECTURE & DESIGN DOCUMENT (EADD)

Version: 2.0.0 · Golden Master Production Blueprint

Target Reviewers: Chief Architects, Technical Directors, Engineering Team Leads, Senior Developers

Date of Architecture Sign-Off: June 20, 2026

Author: Principal Enterprise Systems Architect / Core Technical Lead

1. Executive Summary

CareerForge Pro is an enterprise-grade, high-performance, multi-tenant Software-as-a-Service (SaaS) ecosystem designed to bridge the algorithmic gap between modern job applicants and Applicant Tracking Systems (ATS). The application serves as a comprehensive, closed-loop technical workspace wherein candidates ingest source resumes, parse multi-layered semantic job descriptions (JDs), analyze systemic match scores via deterministic structural metrics, and generate contextually optimal, AI-rewritten professional profiles.

Architecturally, CareerForge Pro leverages a highly optimized MERN stack variants blueprint (React.js, Node.js, Express.js, MongoDB Atlas) specifically refactored to cleanly decouple heavy, resource-intensive operations—such as LLM orchestration, vector tokenizations, and headless browser compilation pools via Puppeteer—from the synchronous HTTP request-response cycle. This ensures constant low-latency profiles across transactional routes under dense usage thresholds.

2. Business Problem

Modern corporate recruiting depends heavily on corporate algorithmic gatekeepers known as Applicant Tracking Systems (ATS), which automatically filter out between 70% and 75% of qualified resumes before human evaluation takes place. The friction originates from three major technical problems:

- **Semantic and Token Mismatch:** Qualified candidates are systematically excluded because their profiles lack the specific programmatic terminology, exact noun arrangements, or domain entities defined in the job description.
- **Structural Layout Distortions:** Legacy parsers fail completely when parsing complex multi-column CSS grid layouts, floating text blocks, custom SVG iconography, or unindexed formatting tables, generating data corruptions inside tracking databases.
- **Tailoring Bottlenecks:** Manually identifying keywords, adjusting phrases, and aligning multiple historical experience metrics across dozens of unique job listings creates an operational bottleneck for job seekers.

3. Project Scope

The product enforces strict architectural execution boundaries to guarantee maintainability and focus system resources on core capabilities:

- **In Scope:** User registration, secure session validation via cryptographically signed HTTP-only state vectors; transactional resume data mutations with multi-versioning state trees; multi-layered TF-IDF matching engines; OpenAI structural schema contract forcing; isolated Puppeteer Chromium browser process pool scaling; Stripe webhooks event state syncing; and an analytics admin console.
- **Out of Scope:** Automated active application browser execution bots (autofill job posting forms), continuous multi-platform job scraper grids, real-time portfolio website hosting services, and automated video/multimedia introduction processing.

4. Software Requirements Specification (SRS)

4.1 Functional Requirements (FR) Matrix

ID	Module	User Role	Technical Specification & Acceptance Criteria
FR-101	Authentication	Guest / User	Must issue stateless JSON Web Tokens safely split/stored in HttpOnly, Secure, and SameSite=Strict cookies.
FR-102	Resume Management	Premium User	Provide comprehensive transactional CRUD capabilities across multiple resume versions (max 10 versions) linked to a single profile entity.
FR-103	ATS Match Analyzer	Premium User	Deconstruct job description strings into domain-specific entities and output a mathematical matching matrix rating from 0 to 100.
FR-104	AI Bullet Optimizer	Premium User	Asynchronously refactor professional summaries via custom prompt mapping structures while strictly preserving historical quantitative metrics.
FR-105	PDF Engine	Premium User	Utilize a resource-pooled, headless Puppeteer instance grid to convert dynamic compiled HTML/CSS documents into standardized searchable print PDFs.
FR-106	Billing Engine	Guest / User	Process card tokens via Stripe Checkout webhooks to automatically adjust localized database authorization structures.
FR-107	Admin Command Panel	Admin	Provide data management capabilities, including the ability to replay failed webhook events, analyze active platform metrics, and suspend user authentication states.

4.2 Non-Functional Requirements (NFR) Specifications

- **Performance Latency:** Standard synchronous API requests must respond within $\leq 200\text{ms}$. Inbound PDF generation requests must complete file compilation, printing, and output streaming within $\leq 3500\text{ms}$.
- **Scalability Target:** The application tier must be completely stateless, allowing frictionless horizontal scaling across container orchestration nodes behind a round-robin load balancer.
- **Security Baselines:** Implement field-level encryption for all sensitive user assets using AES-256-GCM. Require TLS 1.3 across transit pathways, comply with the OWASP Top 10 guidelines, and enforce explicit CORS whitelist origins.
- **Availability & Fault Tolerance:** Target a 99.9% uptime baseline. Implement circuit breakers with short-circuit fallbacks for external AI completions, and use automated daily replication clusters within MongoDB Atlas.

4.3 System Use Case Diagram

SYSTEM USE CASE BOUNDARY MAPPING

```
[ Guest User ] -----> ( Public Landing Pages )
                        |-----> ( Account Sign-Up / Authenticate )

[ Premium User ] ----> ( Manage Multi-Version Resumes )
                        |-----> ( Execute ATS Similarity Scanning )
                        |-----> ( Trigger AI Prompt Bullets Rewrite )
                        |-----> ( Download Puppeteer Vector PDF )
                        |-----> ( Access Stripe Billing Gateway Portal )

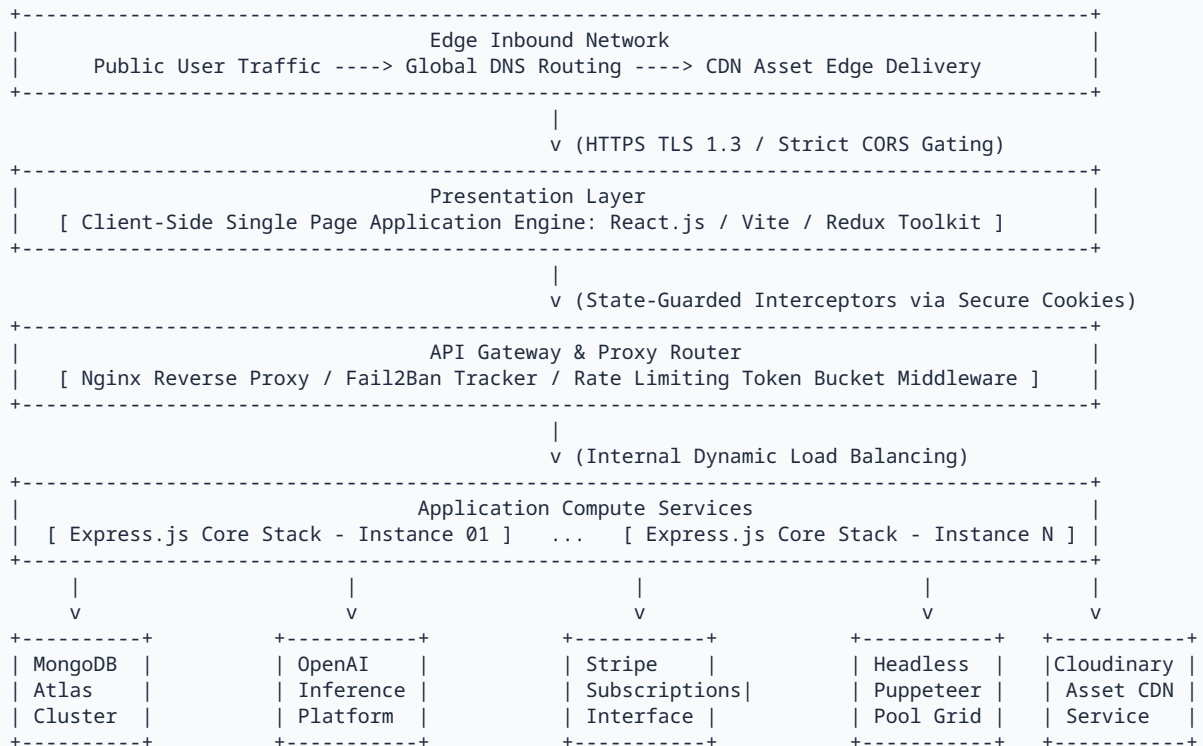
[ Admin Engineer ] --> ( View System Health & Usage Aggregations )
                        |-----> ( Manually Replay Stripe Webhook Logs )
                        |-----> ( Issue Immediate Remote Session Revocations )
```

5. High-Level Design (HLD)

5.1 Multi-Tier System Topography

The global architecture separates presentation states from long-term data store boundaries, running resource-heavy tasks through dedicated, isolated sub-services.

SYSTEM TOPOGRAPHY & CONNECTIVITY TOPOLOGY

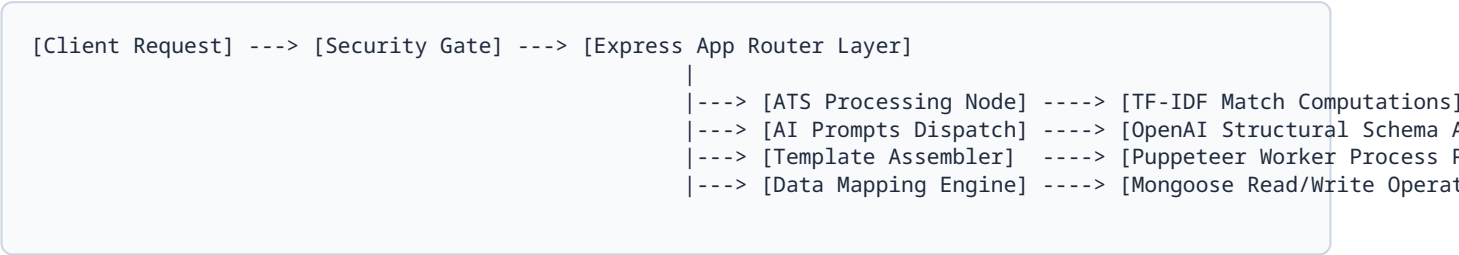


5.2 Data Flow Diagrams (DFD)

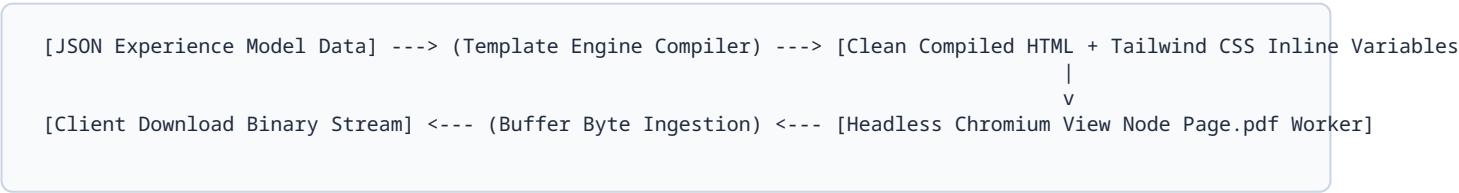
DFD Level 0: Global Context Diagram



DFD Level 1: Sub-System Process Partitioning



DFD Level 2: Detailed Asynchronous Compilation Pipeline



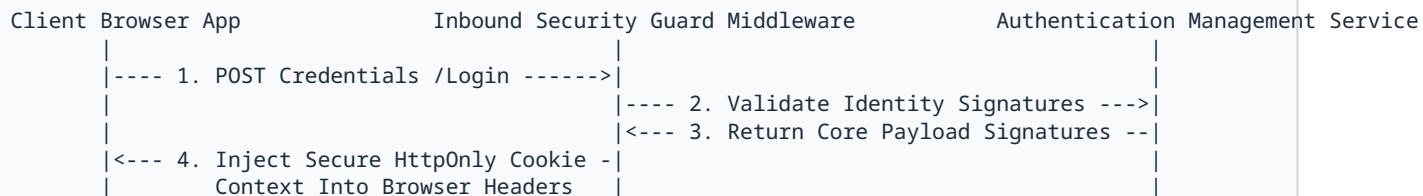
6. Low-Level Design (LLD)

6.1 System Object Class Design Schema

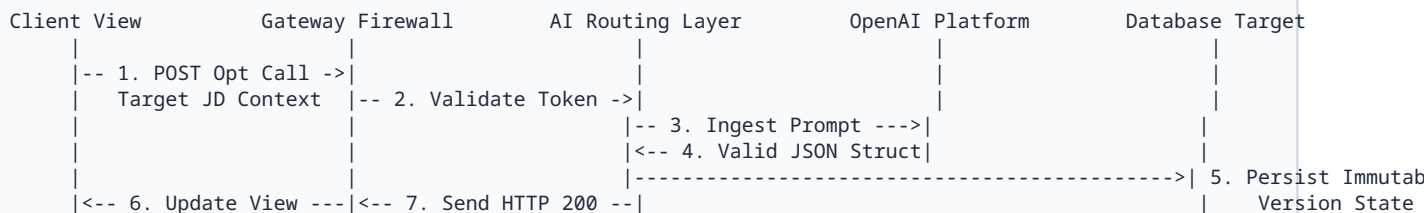


6.2 Architectural Sequence Diagrams

Sequence 1: Client Session Validation Cycle



Sequence 2: End-to-End AI Bullet Point Optimization



7. Database Design & MongoDB Collections

Data persistence uses MongoDB Atlas managed via Mongoose Object Document Modeling (ODM), enforcing indexing strategies on core foreign key identifiers to keep document querying efficient.

7.1 User Configuration Schema Model (`users`)

```
const mongoose = require('mongoose');

const UserSchema = new mongoose.Schema({
  email: {
    type: String,
    required: true,
    unique: true,
    index: true,
    lowercase: true,
    trim: true
  },
  passwordHash: {
    type: String,
    required: true
  },
  role: {
    type: String,
    enum: ['Guest', 'Premium', 'Admin'],
    default: 'Guest',
    required: true
  }
}, { timestamps: true });

module.exports = mongoose.model('User', UserSchema);
```

7.2 Resume Optimization Schema Model (`resumes`)

```
const ResumeSchema = new mongoose.Schema({
  userId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true,
    index: true
  },
  title: {
    type: String,
    required: true,
    trim: true
  },
  personalInfo: {
    name: { type: String, required: true },
    phone: { type: String },
    location: { type: String },
    website: { type: String }
  },
  experience: [{
    company: { type: String, required: true },
    role: { type: String, required: true },
    startDate: { type: Date },
    endDate: { type: Date },
    summary: [{ type: String }] // Sub-bullet arrays for optimization mapping
  }],
  education: [{
    institution: { type: String, required: true },
    degree: { type: String, required: true },
    graduationYear: { type: Number }
  }],
  skills: [{
    type: String,
    index: true
  }],
  currentVersion: {
    type: Number,
    default: 1,
    required: true
  },
  isPremiumTemplate: {
    type: Boolean,
    default: false,
    required: true
  }
}, { timestamps: true });

// Compound index optimized for version scanning lookups
ResumeSchema.index({ userId: 1, currentVersion: -1 });

module.exports = mongoose.model('Resume', ResumeSchema);
```


7.3 Subscription Lifecycle Schema Model (`subscriptions`)

```
const SubscriptionSchema = new mongoose.Schema({
  userId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true,
    unique: true,
    index: true
  },
  stripeCustomerId: {
    type: String,
    required: true,
    index: true
  },
  stripeSubscriptionId: {
    type: String,
    required: true,
    unique: true,
    index: true
  },
  status: {
    type: String,
    enum: ['active', 'canceled', 'past_due', 'incomplete'],
    required: true
  },
  currentPeriodEnd: {
    type: Date,
    required: true
  }
}, { timestamps: true });

module.exports = mongoose.model('Subscription', SubscriptionSchema);
```

8. Detailed Technical Sub-System Design

8.1 Authentication & Authorization Control Middleware

The platform handles session parsing by decoding secure JSON Web Tokens (JWT) extracted from encrypted cookie fields during the request lifecycle.

```

const jwt = require('jsonwebtoken');

const securityAuthenticationGuard = async (req, res, next) => {
  const accountToken = req.cookies.authToken;
  if (!accountToken) {
    return res.status(401).json({ error: "Access Denied: Missing Authorization State Bearer Token" });
  }
  try {
    const verifiedPayload = jwt.verify(accountToken, process.env.JWT_SECRET);
    req.user = { id: verifiedPayload.id, role: verifiedPayload.role };
    next();
  } catch (cryptoError) {
    return res.status(403).json({ error: "Access Denied: Expired or Revoked Crypto Signature Payload" });
  }
};

const restrictToRole = (authorizedRoles) => {
  return (req, res, next) => {
    if (!authorizedRoles.includes(req.user.role)) {
      return res.status(403).json({ error: "Forbidden: Elevated Privilege Scope Clearance Required" });
    }
    next();
  };
};

```

8.2 ATS Vector Match Scoring System

The core analysis engine runs localized tokenization and entity extraction. Raw strings from the user profile and target job description are processed to filter out stop-words, leaving only key domain terms and engineering adjectives.

The mathematical similarity model uses a Term Frequency-Inverse Document Frequency (TF-IDF) representation to construct vector maps. Let the parsed resume tokens be represented as a vector space map R and the job description constraints be mapped as a targeting vector space J . The system calculates the inner dot product normalized by the product of their Euclidean norms to determine the final match value:

$$\text{ATS Match Score} = \left(\frac{R \cdot J}{\|R\| \|J\|} \right) \times 100$$

8.3 AI Prompt Engineering & Output Gating Strategy

To prevent model hallucination and preserve formatting across versions, the platform sets rigid structural rules within OpenAI execution contexts.

```

const configureAIInferencePayload = (resumeBulletPoints, jobDescriptionText) => {
  return {
    model: "gpt-4o",
    temperature: 0.1, // Highly deterministic sampling
    response_format: {
      type: "json_schema",
      json_schema: {
        name: "bullet_optimization_response",
        schema: {
          type: "object",
          properties: {
            optimizedBullets: {
              type: "array",
              items: { type: "string" },
              description: "Action-oriented rewritten phrases integrating target job keywords."
            }
          },
          required: ["optimizedBullets"],
          additionalProperties: false
        }
      }
    },
    messages: [
      {
        role: "system",
        content: "You are an enterprise ATS resume optimizer. Rewrite user experience bullets to align with the target job description. Incorporate target keywords naturally. Do not fabricate metrics, company titles, or historical credentials."
      },
      {
        role: "user",
        content: `Target Job Description: ${jobDescriptionText}
Source Bullet Points: ${JSON.stringify(resumeBulletPoints)}`
      }
    ]
  };
};

```

8.4 Stripe Payment Hook Verification Flow

Payment state synchronization relies entirely on processing asynchronous webhook events signed directly by Stripe infrastructure.

```

const handleStripeWebhookInbound = async (req, res) => {
  const incomingSignature = req.headers['stripe-signature'];
  let structuredEventPayload;

  try {
    structuredEventPayload = stripe.webhooks.constructEvent(
      req.body,
      incomingSignature,
      process.env.STRIPE_WEBHOOK_SECRET
    );
  } catch (err) {
    return res.status(400).send(`Webhook Signature Authentication Failure: ${err.message}`);
  }

  if (structuredEventPayload.type === 'checkout.session.completed') {
    const activeSession = structuredEventPayload.data.object;
    // Transactional role assignment mapping user ID out of session metadata
    await User.findByIdAndUpdate(activeSession.metadata.userId, { role: 'Premium' });
  }

  res.status(200).json({ received: true });
};

```

8.5 Headless Puppeteer Resource Isolation Architecture

To protect backend processes from memory leaks and CPU exhaustion common with headless browsers, the system isolates printing workflows inside a thread-safe generic worker pool.

```

const pool = require('generic-pool');
const puppeteer = require('puppeteer');

const browserClusterPool = pool.createPool({
  create: async () => {
    return await puppeteer.launch({
      headless: true,
      args: [
        '--no-sandbox',
        '--disable-setuid-sandbox',
        '--disable-dev-shm-usage',
        '--disable-gpu'
      ]
    });
  },
  destroy: async (browserInstance) => {
    await browserInstance.close();
  }
}, {
  min: 2,
  max: 10,
  idleTimeoutMillis: 30000
});

const renderResumeToVectorPdfStream = async (compiledHtmlContent) => {
  const workingBrowser = await browserClusterPool.acquire();
  const renderingPage = await workingBrowser.newPage();
  try {
    await renderingPage.setContent(compiledHtmlContent, { waitUntil: 'networkidle0' });
    const generatedPdfBufferBytes = await renderingPage.pdf({
      format: 'A4',
      printBackground: true,
      margin: { top: '15mm', right: '15mm', bottom: '15mm', left: '15mm' }
    });
    return generatedPdfBufferBytes;
  } finally {
    await renderingPage.close();
    await browserClusterPool.release(workingBrowser);
  }
};

```

9. Folder Structure & API Specifications

9.1 Production Repository Topography Layout

```
Ats-Resume-Builder
├── client/                    # Presentational Single Page Architecture Root
│   ├── public/              # Static Production Graphics & Standard Manifests
│   └── src/
│       ├── components/      # Shared Functional Atomic Visual Elements (ShadCN UI)
│       ├── pages/           # Isolated Layout Views (Dashboard, Builder Workspace)
│       ├── store/            # Centralized App State Slices (Redux Toolkit)
│       └── services/         # Axios Structural Interceptor Declarations
└── server/                  # Stateless Application Compute Container
    ├── src/
    │   ├── config/          # Connection Clients Ingestion (MongoDB Atlas, Stripe SDK)
    │   ├── controllers/     # Payload Extraction & Route Action Orchestration
    │   ├── models/          # Mongoose Domain Schema Blueprint Architecture Definitions
    │   ├── middleware/      # Security Isolation Filters, Cors Rules, and Rate Limiters
    │   └── services/        # Decoupled Integration Modules (OpenAI, Puppeteer Pool)
    ├── app.js               # Operational Middleware Configuration Pipeline Entry
    └── server.js            # Cluster Bootstrapping & Hardware Port Ingestion Listener
```

9.2 Complete API Route Specifications

Method	Path Endpoint	Required Scope	Data Interface & Target System Logic Contract
POST	`/api/v1/auth/register`	Public	Accepts raw candidate identity properties; saves encrypted hash to datastore; issues session cookie.
POST	`/api/v1/auth/login`	Public	Validates structural password strings; issues secure HTTP-Only authorization tokens.
GET	`/api/v1/resumes`	Authenticated User	Fetches all multi-version resume items associated with the requesting user context.
POST	`/api/v1/analyze/ats`	Premium Scope	Accepts target JD text payloads and computes vector match score metrics.
POST	`/api/v1/optimize/rewrite`	Premium Scope	Triggers OpenAI context builder to optimize resume bullet structures using target keywords.
POST	`/api/v1/export/pdf`	Authenticated User	Renders HTML content within a headless browser instance and downloads the compiled file bytes.
GET	`/api/v1/admin/analytics`	Admin Elevate	Aggregates platform KPIs across multi-tenant database records for infrastructure monitoring.

10. DevOps, Security, and Scalability Management

10.1 Production Infrastructure Environment Manifest (`.env`)

```
# Server Core Configuration Parameters
NODE_ENV=production
PORT=5000

# High-Availability Datastore Configuration Target
MONGO_URI=mongodb+srv://forge_cluster_root:super_secure_vault_hash_2026@cluster0.mongodb.net/forge_prod?retryWrites=true&w=majority

# Cryptographic Signatures Framework
JWT_SECRET=f3948ba98c11e3092bb2138940561e1b1284a1d5093a201c27bf8419cbda52c1fe41

# Enterprise Integration Gateway Providers
STRIPE_SECRET_KEY=sk_live_51P2348912BCDEF982341
STRIPE_WEBHOOK_SECRET=whsec_923841092384abcdef1092348
OPENAI_API_KEY=sk-proj-401239abfcd129348102394812039481023948
CLOUDINARY_URL=cloudinary://981239812:secret_hash@careerforge-pro
```

10.2 Continuous Integration & Deployment (CI/CD) Blueprint

The automated development lifecycle enforces code quality checks and validation steps before deployment.

- **Commit Hook Gating:** Automatically triggers localized compilation health assessments and runs standard Jest unit test suites on push events.
- **Containerization Blueprint:** Builds and tags lean, multi-stage production Docker images that encapsulate the required headless Chromium dependencies.
- **Staging Integration Testing:** Deploys container assets to isolated staging clusters and runs automated Playwright verification scripts across core user paths.
- **Zero-Downtime Rollout:** Executes a canary or blue-green release strategy, switching load balancer traffic to the new cluster configuration once all container health checks pass.

10.3 Production Security Architecture Implementation

```
const helmet = require('helmet');
const rateLimit = require('express-rate-limit');

const initializeSecurityShields = (expressAppInstance) => {
  // Enforce rigid secure headers via Helmet to guard against common script injection and
  // clickjacking vectors
  expressAppInstance.use(helmet({
    contentSecurityPolicy: {
      directives: {
        defaultSrc: ["'self'"],
        scriptSrc: ["'self'", "'trusted-cdn.com'"],
        styleSrc: ["'self'", "'unsafe-inline'"],
        imgSrc: ["'self'", "data:", "res.cloudinary.com"]
      }
    }
  }));

  // Restrict high-frequency endpoint actions across public paths
  const publicAuthRateLimiter = rateLimit({
    windowMs: 15 * 60 * 1000, // 15 Minute Evaluation Window
    max: 100, // Bound individual IP footprints to 100 calls per frame
    message: { error: "Transaction verification threshold exceeded. Please retry after some time." }
  });

  expressAppInstance.use('/api/v1/auth/', publicAuthRateLimiter);
};
```

10.4 Scalability Architecture Overview

Because rendering PDF layouts via headless browser engines requires significant CPU resources, the deployment strategy isolates the Puppeteer execution environment from the main Express application threads. High-volume export requests are handled via a managed job queue system, preventing single-thread blocking issues and ensuring stable response times across the primary platform API.

SCALABILITY STRATEGY VIEW



11. Quality Assurance, Risks, & Development Roadmap

11.1 Comprehensive Verification Strategy

- **Unit Testing Suite:** Employs Jest to validate the text tokenization calculations, TF-IDF vector parsing math, and schema parsing transformations.
- **Integration Testing:** Verifies API endpoint parameters, token authorization state changes, and error responses under simulated database disconnects.
- **End-to-End (E2E) Automation:** Uses Playwright scripts to simulate complete multi-tier user paths, from initial registration to premium Stripe upgrades and document downloads.

11.2 Development Phase Planning

- **Phase 1 (Weeks 1-2):** Core database construction, Mongoose mapping definitions, and secure cookie-based session authorization routines.
- **Phase 2 (Weeks 3-4):** Integration of the TF-IDF matching matrix along with specialized prompt schema validation modules for OpenAI completions.
- **Phase 3 (Weeks 5-6):** Setup of the pooled Puppeteer rendering engine alongside the signature-verified Stripe subscription billing endpoints.
- **Phase 4 (Weeks 7-8):** Production security hardening, end-to-end performance benchmarking, and multi-region cluster deployment.

11.3 Architectural Risk Analysis Matrix

Identified Risk Event	Impact Rating	Architectural Mitigation Plan & Resolution Tactics
ATS Format Shifts	Medium	Enforce a strict single-column print layout that avoids visual tables and complex CSS styling to ensure standard text parsers can read the content cleanly.
AI Provider Latency / Outages	High	Implement the circuit-breaker pattern with short-circuit thresholds (e.g., 8-second execution timeouts) and provide helpful error responses to the client.
Browser Pool Memory Leaks	High	Use generic-pool strategies to set strict request limits per browser instance, automatically recycling Chromium processes once they hit their maximum usage threshold.

12. Conclusion

The unified architecture presented in this document provides a comprehensive blueprint for building a secure, performant, and reliable SaaS platform. By isolating resource-heavy operations like document printing and AI generation from the main application thread, CareerForge Pro is well-positioned to maintain consistent performance and meet enterprise scalability standards.